

## Bài 12: Thêm chức năng Bình luận (Comments) cho News

### Mục tiêu

- Tạo migration cho bảng comments với các cột: id, news\_id, title, content, author\_id, created\_at, updated\_at.
- Tạo model Comment và thiết lập quan hệ với News (và User nếu có).
- Tạo CommentController để lưu và hiển thị bình luận.
- Tạo CommentSeeder để sinh dữ liệu mẫu.
- Tạo các view Blade để hiển thị danh sách bình luận và form gửi bình luận dưới mỗi bài viết.

### Bước 1: Tạo migration cho bảng comments

Chạy lệnh sau để tạo migration:

```
php artisan make:migration create_comments_table --create=comments
```

Mở file migration vừa tạo trong **database/migrations** và thay nội dung method **up()** như sau:

```
public function up()
{
    Schema::create('comments', function (Blueprint $table) {
        $table->id();
        $table->foreignId('news_id')->constrained('news')->onDelete('cascade');
        $table->string('title')->nullable();
        $table->text('content');
        $table->foreignId('author_id')->nullable()->constrained('users')-
>nullOnDelete();
        $table->timestamps();
    });
}
```

Giải thích:

- news\_id: liên kết tới trường id trong bảng news (khi xóa bài viết, các bình luận sẽ bị xóa theo - cascade).
- title: tiêu đề bình luận (có thể để NULL nếu không cần).
- content: nội dung bình luận.
- author\_id: tham chiếu tới users.id, nullable để cho phép khách bình luận; khi user bị xóa, author\_id sẽ thành NULL (tương đương với việc cho phép khách – guest bình luận).

### Bước 2: Chạy migration

Chạy lệnh để thực thi migration:

```
php artisan migrate
```

[Chụp hình ảnh kết quả vào đây]

### Bước 3: Tạo Model Comment và khai báo quan hệ

Tạo model (tùy chọn tạo factory):

**php artisan make:model Comment**

Mở file **app/Models/Comment.php** và cập nhật nội dung:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Comment extends Model
{
    //use HasFactory;

    protected $fillable = [
        'news_id',
        'title',
        'content',
        'author_id',
    ];

    public function news()
    {
        return $this->belongsTo(News::class);
    }

    public function author()
    {
        return $this->belongsTo(User::class, 'author_id');
    }
}
```

Giải thích: Phương thức `belongsTo()` được dùng trong **Model con** (bảng có chứa khóa ngoại) để khai báo mối quan hệ *thuộc về* một Model khác.

Cập nhật model News để thêm quan hệ comments (mở **app/Models/News.php**):

```
public function comments()
{
```

```
    return $this->hasMany(Comment::class);
}
```

Giải thích: Phương thức `hasMany()` được dùng trong **Model cha (parent model)** để khai báo rằng "Một bản ghi của model này có thể có **nhiều bản ghi liên quan** trong model kia." Qua đó giúp lấy bình luận của một bài viết bằng `$news->comments()`.

#### Bước 4: Tạo CommentSeeder (dữ liệu mẫu)

Tạo seeder:

```
php artisan make:seeder CommentSeeder
```

Mẫu nội dung CommentSeeder sử dụng Faker:

```
<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;
use App\Models\News;
use App\Models\Comment;
use Faker\Factory as Faker;

class CommentSeeder extends Seeder
{
    public function run(): void
    {
        $faker = Faker::create();

        $newsList = News::all();

        foreach ($newsList as $news) {
            $count = rand(2, 6); // mỗi bài 2-6 comment
            for ($i = 0; $i < $count; $i++) {
                Comment::create([
                    'news_id' => $news->id,
                    'title' => $faker->optional()->sentence(3),
                    'content' => $faker->paragraph(2),
                    'author_id' => null, // hoặc rand user id nếu có users
                ]);
            }
        }
    }
}
```

```
}  
}
```

Đăng ký seeder trong DatabaseSeeder.php hoặc chạy trực tiếp:

```
php artisan db:seed --class=CommentSeeder
```

[Chụp hình ảnh kết quả vào đây]

### Bước 5: Tạo CommentController

Tạo controller resource cho comment:

```
php artisan make:controller CommentController --resource
```

[Chụp hình ảnh kết quả vào đây]